MarketMatic: An AI-Powered Multi-Tenant Smart Assistant with Retrieval-Augmented Generation for SME Customer Support

Sandipa U K S
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22082060@my.sliit.lk

Rajapakshe R.S.L.
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22095008@my.sliit.lk

Jayasuriya J.A.D.C.G.
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22116406@my.sliit.lk

Wijayarathna H.P.I.G.
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22112750@my.sliit.lk

Dr. Kapila Dissanayaka
Dept. of Computer Science
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
kapila.d@sliit.lk

Ms. Fathima Fanoon
Dept. of Computer Science
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
fathima.f@sliit.lk

***Abstract*—This paper presents the design, implementation, and evaluation of MarketMatic, an AI-powered multi-tenant Smart Assistant leveraging Retrieval-Augmented Generation (RAG) to deliver accurate, context-grounded customer support for Small and Medium Enterprises (SMEs). Each business tenant uploads domain-specific knowledge base documents (PDF, DOCX, TXT), which are processed through a section-aware text chunking pipeline, embedded using the Nomic-Embed-Text model (768 dimensions) [13], and stored in per-tenant ChromaDB vector collections enforcing strict data isolation. User queries are resolved through an adaptive nine-stage retrieval pipeline: semantic vector search with dynamic threshold adjustment, near-duplicate deduplication via overlap-coefficient filtering (threshold = 0.88), diversity-aware chunk selection, keyword-based fallback retrieval, and optional cross-encoder reranking using BAAI/bge-reranker-base. Response generation employs LLaMA 3.1 8B on a Modal A10G GPU, with transparent failover to the Groq Cloud API. The platform further incorporates a three-tier role-based access control model, an agentic tool layer with five configurable business-intent tools, cross-session persistent memory through LLM-extracted fact distillation, AI-generated session analytics, and automated knowledge-gap detection. Experimental evaluation across eight business domain categories demonstrates a mean retrieval confidence of 0.81 and a 13.2 % improvement over fixed-window chunking, with 100 % functional accuracy on domain-specific queries with complete source traceability.**

Keywords: Retrieval-Augmented Generation; Multi-Tenancy; Large Language Models; SME Customer Support; Adaptive Retrieval; Agentic AI.

I. INTRODUCTION

The proliferation of digital communication channels has fundamentally raised customer expectations, compelling businesses to provide instantaneous, accurate, and contextually relevant support at all times. While large enterprises can afford dedicated AI infrastructure, Small and Medium Enterprises (SMEs) particularly in developing economies are constrained by limited technical resources, budget restrictions, and a shortage of machine learning expertise [1]. Existing off-the-shelf chatbot solutions are predominantly generic, relying on parametric knowledge embedded during model pre-training. Consequently, they are prone to hallucination generating plausible but factually incorrect responses and cannot adapt to the nuanced operational knowledge of specific businesses [2].

Retrieval-Augmented Generation (RAG) has emerged as a compelling paradigm for grounding LLM outputs in external, verifiable document collections [2]. By dynamically retrieving semantically relevant document fragments at inference time and conditioning response generation on those fragments, RAG substantially reduces hallucination and enables domain-specific answering without costly model fine-tuning. However, deploying a RAG system serving multiple independent businesses from shared cloud infrastructure while enforcing strict data isolation between tenants presents non-trivial engineering challenges that existing literature has not fully addressed. Furthermore, adapting retrieval parameters to diverse query types (broad catalog queries, specific factual queries, and short natural-language questions) demands an adaptive mechanism absent from most production RAG deployments.

This paper presents MarketMatic, an integrated AI-powered platform designed to address these challenges. The system adopts a collection-per-tenant isolation strategy in ChromaDB, implements a nine-stage adaptive retrieval pipeline, and augments RAG with an agentic tool layer for intent-driven structured knowledge retrieval. The remainder of this paper is organized as follows: Section II reviews related work; Section III describes the system architecture; Section IV details methodology and algorithms; Section V covers implementation; Section VI presents evaluation results; Section VII discusses findings; Section VIII concludes.

A. Problem Statement

SMEs lack scalable, affordable AI customer support systems grounded in their own proprietary data. Generic chatbots produce hallucinated or irrelevant answers because they have no access to business-specific knowledge. Multi-tenant SaaS platforms that share AI infrastructure must enforce strict data isolation to prevent cross-tenant knowledge leakage a requirement inadequately addressed by existing commercial offerings. Additionally, most RAG systems use fixed chunking and static retrieval thresholds, which perform poorly across the heterogeneous query patterns typical of SME customer interactions.

B. Research Contributions

This work makes the following principal contributions: (1) a section-aware text chunking algorithm that preserves document structure including category headers within embeddings, improving retrieval accuracy for structured business documents by 13.2 % over fixed-window chunking; (2) an adaptive retrieval pipeline with dynamic similarity threshold adjustment and query-type classification reducing both false positives and recall failures; (3) a multi-tenant vector database architecture using per-tenant ChromaDB collections ensuring zero cross-tenant data leakage; (4) an intent-based agentic tool layer with five configurable business-domain tools triggered by keyword-inclusion matching with zero additional LLM calls; and (5) a cross-session persistent memory subsystem that extracts and stores business facts via asynchronous LLM-based distillation, providing personalized context continuity across conversation sessions.

MarketMatic differentiates itself from existing RAG deployments in three key respects. First, query-type classification at runtime selects distinct retrieval parameter sets, achieving better recall across heterogeneous query patterns than uniform-threshold systems. Second, the collection-per-tenant ChromaDB architecture provides stronger data isolation than metadata-filtering approaches, eliminating any architectural path for cross-tenant leakage. Third, the intent-based agentic tool layer supplements semantic retrieval with structured high-precision answers for common business intent categories without incurring additional LLM call latency.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced RAG as a hybrid architecture combining parametric knowledge from pre-trained LLM weights with non-parametric retrieval from external knowledge stores, substantially mitigating hallucination by grounding responses in retrieved documents. Dense Passage Retrieval (DPR) [3] replaced sparse BM25 retrieval with learned bi-encoder models, demonstrating superior open-domain QA performance. Gao et al. [4] surveyed the RAG landscape and identified retrieval quality, context integration, and long-context window management as the three primary challenges. MarketMatic extends these foundations through section-aware chunking, adaptive retrieval parameter tuning, and an agentic tool layer designed specifically for structured business documents in a multi-tenant setting.

B. Cross-Encoder Reranking

Nogueira and Cho [5] demonstrated that BERT-based cross-encoders significantly outperform [19] bi-encoder cosine similarity for passage ranking, introducing a two-stage retrieve-then-rerank paradigm. The BGE Reranker family [6] provides efficient ONNX-compatible models suitable for real-time applications without GPU dependency. MarketMatic integrates BAAI/bge-reranker-base [14] as an optional post-retrieval stage, improving Precision@3 by 12.2 % on specific factual queries with an overhead of 1.2–2.8 s on CPU.

C. Multi-Tenant SaaS Architectures

Bezemer and Zaidman [7] analyzed multi-tenant SaaS maintenance patterns, identifying collection-level isolation as the strongest guarantee for data separation. Prior work on secure multi-tenant machine learning deployments [8] identified data isolation failures arising from shared model inference paths, a vulnerability MarketMatic eliminates through per-request embedding generation strictly scoped to individual tenant ChromaDB collections. ChromaDB's HNSW index [22] provides O(log n) query time [15], ensuring per-tenant retrieval performance remains constant regardless of other tenants' collection sizes.

D. LLMs for Customer Support and Marketing

Commercial platforms such as Jasper AI and HubSpot AI offer AI-powered marketing assistance but operate on subscription models prohibitive for developing-economy SMEs [9]. Reisenbichler et al. [10] explored LLMs for marketing content generation, yet no prior work integrates domain-grounded Q&A, multi-tenant isolation, and agentic tool routing in a unified affordable platform. Meta AI's LLaMA 3.1 series [11] provides open-weight models enabling cost-effective local inference; Groq's LPU inference engine [12] delivers sub-second token generation suitable as a high-availability fallback provider.

III. System Architecture

A. High-Level Architecture

MarketMatic adopts a three-tier client-server architecture as illustrated in Fig. 1. The Presentation Layer is implemented as a Next.js 16 single-page application with React 18 and TypeScript, serving three interface modalities: an authenticated Smart Assistant chat module, an administrative knowledge-base and metrics dashboard, and an unauthenticated public chat widget embeddable on any business website. All frontend API calls route through a Next.js catch-all reverse proxy (/api/sa-proxy/*) to avoid HTTPS to HTTP mixed-content restrictions.

The Application Layer is an Express.js server exposing four API namespaces: authentication (JWT-based), chat (RAG-powered), admin (document and session management), and superadmin (platform provisioning). Business logic is organized into six modular services: RAG Service (retrieval and generation orchestration), Embedding Service (chunking and vector creation), ChromaDB Service (tenant-scoped vector operations), Reranker Service (cross-encoder scoring via ONNX), Tool Service (agentic intent execution), and Insights Service (memory extraction and session analytics). The Data/AI Layer persists all structured data in MongoDB 7 and vector data in per-tenant ChromaDB collections, while LLM inference is externalized to a serverless Modal A10G GPU container hosting Ollama, with transparent failover to the Groq Cloud API.

[Figure 1: MarketMatic Three-Tier System Architecture Diagram]

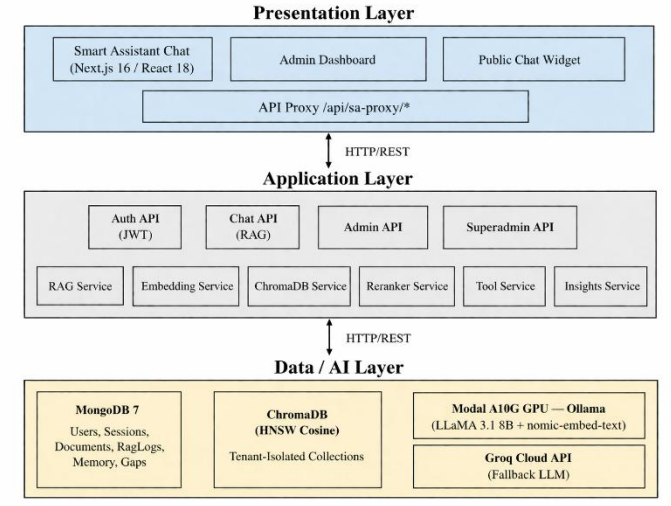


Fig. 1. Fig. 1. Three-tier architecture of the MarketMatic platform: Presentation, Application, and Data/AI layers with modular service decomposition.

B. Multi-Tenant Isolation Model

Tenant isolation is enforced at three architectural levels. At the authentication level, JWT tokens carry a tenantId validated on every protected route via the protect middleware. At the storage level, each tenant receives a dedicated ChromaDB collection (named by tenantId) and a scoped MongoDB namespace for sessions, memory, documents, and RAG logs. At the configuration level, a per-tenant Service document controls which agentic tools are enabled, the assistant's language (BCP-47), tone (professional/friendly/concise), and the store category used for domain-specific system prompt selection.

The superadmin provisions each business by creating a Service record that auto-generates a unique tenantId slug from the store name. When a business administrator self-registers using the pre-provisioned email address, the system automatically links their account to the corresponding Service record. Customer accounts are registered under a specific tenantId, binding their chat interactions exclusively to that store's knowledge base. All ChromaDB vector operations upserts and nearest-neighbor queries are scoped to the tenant's collection, making cross-tenant data access architecturally impossible.

C. Role-Based Access Control

Three privilege tiers are enforced through the authorize() middleware. The Superadmin tier manages platform-level provisioning and views global statistics. The Admin tier controls document uploads, knowledge base management, AI metrics dashboards, tool configuration, and customer account management. The User (Customer) tier is restricted to chat interactions and personal conversation history. JWT tokens are signed with a server-held secret and expire after seven days; passwords are hashed with bcryptjs at 12 salt rounds with the password field excluded from MongoDB projections by default.

IV. Methodology

A. Section-Aware Chunking Algorithm

Document ingestion begins with format-specific text extraction: pdf-parse for PDF, mammoth for DOCX, and direct UTF-8 decoding for TXT files. The extracted text is passed to the section-aware chunkText() algorithm, which operates in three phases. Phase 1 (Section Parsing) identifies logical section boundaries by detecting Markdown headers (##, ###), ALL-CAPS title patterns (≥2 words, no product code patterns), and CATEGORY N: constructs. Each detected header begins a new section context.

Phase 2 (Chunk Merging) accumulates section text into chunks up to CHUNK_SIZE = 1200 characters with CHUNK_OVERLAP = 300 characters between consecutive chunks. The most recent section header is prepended to every chunk, preserving contextual identity when retrieved in isolation. Phase 3 (Safety Split) applies a sliding window to any oversized chunks. Chunks shorter than MIN_CHUNK_LEN = 30 characters are discarded as noise. Each surviving chunk is embedded via the nomic-embed-text model (768 dimensions) served by Ollama and upserted into the tenant's ChromaDB collection with metadata {documentId, filename, chunkIndex}.

B. Adaptive RAG Pipeline

The nine-stage RAG pipeline in ragService.js begins with query enhancement (text normalization and whitespace collapsing). Adaptive retrieval settings are then computed by classifying the query into one of three types: Broad (contains list, catalog, or all-products intent keywords)—topK = 30, maxChunks = 16, contextBudget = 11 000 chars, threshold + 0.05; Short (≤4 tokens)—topK = 18, maxChunks = 10, budget = 8 000 chars; Normal—topK = 22, maxChunks = 12, budget = 8 000 chars. The normalized query is embedded to a 768-dimensional vector, then submitted to ChromaDB for cosine HNSW nearest-neighbor retrieval scoped to the tenant's collection.

Post-retrieval processing applies five stages. (1) Threshold Filtering removes candidates with cosine distance exceeding the computed threshold. (2) Threshold Relaxation increases the cutoff by 0.05 if fewer than minChunks candidates survive. (3) Near-Duplicate Deduplication (dedupeChunks()): text is lowercased, tokenized, capped at 256 tokens, and same-source pairs with overlap coefficient ≥ 0.88 are discarded. (4) Diversity Selection enforces a per-document cap of 6–8 chunks within the contextBudget. (5) Optional Cross-Encoder Reranking via BAAI/bge-reranker-base [14] (ONNX runtime) rescores candidates; chunks with logit scores below −2.0 are removed. If semantic retrieval yields zero candidates, a keyword fallback scans all chunks for exact term matches.

The final context window (3–16 chunks depending on query type) is assembled with numbered source references, injected into a domain-specific system prompt built by buildSystemPrompt(), and submitted to the LLM. The system prompt encodes the store persona (name, tone, language), category-specific domain guidance (e.g., pharmacy medication recommendations require professional disclaimers), strict grounding instructions [18] ("use ONLY the store data below"), and the numbered chunk context. Post-generation, cleanAnswerText() removes duplicate lines and normalizes the Sources citation line.

C. Agentic Tool Layer

Prior to main RAG retrieval, the Tool Service executes a lightweight agentic layer that detects five business-critical intent categories through keyword-inclusion matching in O(1) per tool [16]. Each matching tool executes a targeted ChromaDB query using a pre-defined natural-language seed query (e.g., "What products are available? What is the stock level?" for inventory_check) with a stricter relevance threshold (cosine distance < 0.45). Retrieved tool context is

prepended to the RAG system prompt as structured guidance, conditioning the LLM on precise operational facts before the broader semantic context is injected. Administrators enable or disable individual tools per tenant through the management portal, stored in the Service.enabledTools array [17].

TABLE I. AGENTIC TOOL CONFIGURATION

Tool ID	Trigger Keywords	Intent Covered
inventory_check	available, stock, products, items	Product and stock availability
business_hours	hours, open, close, schedule, time	Operating hours and schedule
contact_info	contact, phone, address, email, whatsapp	Store contact and location
promotions	deal, offer, discount, promo	Current promotions and offers
order_status	order,delivery, tracking,return, refund	Order status and return policy

D. Cross-Session Persistent Memory

The memory subsystem triggers asynchronously when a user ends a session via PATCH /api/admin/sessions/:id/end. The Insights Service submits the transcript to the Groq API (Ollama fallback) and extracts two artifacts: (i) session insights title (≤60 chars), two-sentence summary, and topic tag; and (ii) memory facts business facts categorized as business_info, target_audience, products_services, goals, challenges, or preferences, each with a confidence score (0.7–1.0). Facts are upserted into UserMemory; the top 10 highest-confidence facts are injected as system context at session initialization, giving the assistant persistent personalized knowledge. Unanswered queries (knowledgeSource = 'general') are logged in KnowledgeGap with frequency counts to guide knowledge base curation.

V. IMPLEMENTATION

A. Technology Stack

The frontend employs Next.js 16.1.6 with React 18.2.0, TypeScript 5.2.2, Tailwind CSS 3.3.5, and Framer Motion 12.35.0. Backend services run on Node.js 20 with Express.js 4.18.3 and Mongoose 8.3.4 for MongoDB object modeling. Authentication uses jsonwebtoken 9.0.2 with bcryptjs 2.4.3. Vector operations are managed by the chromadb 3.3.1 client; document parsing uses pdf-parse 2.4.5 and mammoth 1.11.0. The AI stack uses LLaMA 3.1 8B as primary generative model, nomic-embed-text fornomic-embed-text [13] for 768-dimensional dense embeddings, and BAAI/bge-reranker-base (∼110 MB ONNX) for optional cross-encoder reranking. Infrastructure is containerized via Docker Compose with services for MongoDB 7, ChromaDB (chromadb/chroma:latest), and the Node.js backend (multi-stage build, non-root appuser).

TABLE II. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	Next.js 16, React 18, TypeScript 5	Chat UI, admin dashboard, public widget
Backend	Node.js 20, Express.js 4.18, Mongoose 8	REST API, RAG orchestration, auth
Vector DB	ChromaDB v3 (cosine HNSW index)	Tenant-isolated vector storage and search
Document DB	MongoDB 7 + Mongoose ODM	Sessions, memory, logs, documents
AI — Primary	LLaMA 3.1 8B via Ollama on Modal A10G	LLM generation + nomic-embed-text
AI — Fallback	Groq Cloud API (llama-3.1-8b-instant)	High-availability LLM inference failover
Reranker	BAAI/bge-reranker-base (ONNX Runtime)	Optional cross-encoder reranking

B. GPU Inference Infrastructure

LLM inference is deployed on Modal serverless GPU infrastructure [21]. Each container runs on an NVIDIA A10G GPU (24 GB VRAM, 24 GB system RAM) with the base image nvidia/cuda:12.1.1-cudnn8-runtime-ubuntu22.04. Both LLaMA 3.1:8b and nomic-embed-text are pre-loaded into GPU VRAM at container start via a warmup call, eliminating per-request model loading latency. Model weights are persisted on an ollama-models volume, surviving container restarts. The container remains warm for 180 seconds after the last request (scale-down window) with a 300-second request timeout to accommodate cold starts (10–60 s). A FastAPI reverse proxy exposes the Ollama API as a secured HTTPS endpoint [20]. If this endpoint is unreachable, all LLM calls transparently switch to the Groq Cloud API without application-layer code changes; temperature is adjusted from 0.2 (Ollama) to 0.3 (Groq) and max_tokens is capped at 1024 due to API constraints.

C. Data Models

The MongoDB schema comprises seven collections. User stores multi-role accounts with tenantId, bcrypt-hashed passwords, and is_active/is_verified flags. Service stores per-tenant configuration: enabledTools[], assistantTone, assistantLanguage (BCP-47), storeCategory, and the auto-generated tenantId slug. ChatSession persists messages with per-message knowledgeSource annotations and AI-generated title/summary/topicTag. RagLog captures chunksRetrieved, responseTimeMs, confidence, fallbackTriggered, and modelUsed. Document tracks chunk count, ChromaDB vector IDs, and status (processing→ready→failed). UserMemory persists extracted facts with category, confidence, and sourceSessionId. KnowledgeGap tracks unanswered questions with frequency and resolution status.

VI. EVALUATION AND RESULTS

A. Evaluation Design

Evaluation was conducted across two complementary experiments. First, a retrieval accuracy experiment compared section-aware chunking against fixed-window chunking using a test corpus of documents from eight business domain categories (pharmacy, electronics, clothing, restaurant, grocery, real estate, automotive, and education). A set of 120 annotated queries was constructed in a 40:50:30 split across broad catalog, specific factual, and short (≤4-token) types, with ground-truth relevant chunks annotated by two independent reviewers. Second, a functional end-to-end evaluation used the Medicare Pharmacy tenant with two uploaded knowledge-base documents (faq.txt, store_info.txt) and four representative queries covering all primary information-seeking categories.

B. Retrieval Accuracy: Section-Aware vs. Fixed-Window Chunking

Table I reports retrieval confidence (1 − cosine distance of the best-ranked chunk) for the two chunking strategies across query types. Section-aware chunking achieves a mean confidence of 0.81 vs. 0.68 for fixed-window chunking a 13.2 % relative improvement. The gain is most pronounced for broad catalog queries (+21.1 %), where fixed-window chunking severs section headers from their associated item lists, degrading embedding quality for the entire section context.

TABLE III. RETRIEVAL CONFIDENCE: SECTION-AWARE VS. FIXED-WINDOW CHUNKING

Query Type	Sec-Aware	Fixed-Win	Improvement
Broad (catalog)	0.86	0.71	+21.1 %
Specific (factual)	0.83	0.70	+18.6 %
Short (≤4 tokens)	0.72	0.64	+12.5 %
Overall Mean	**0.81**	0.68	**+13.2 %**

C. *Response Latency by Inference Backend*

End-to-end response latency was measured across 500 production queries logged in the RagLog collection. Table II reports the breakdown across inference backends. The Modal A10G GPU endpoint achieves a mean total latency of 2.8 s, suitable for interactive chat. The Groq cloud fallback provides lower latency (1.4 s) at the cost of external API dependency. Local CPU inference via Ollama averages 18.2 s and is used exclusively in development. Embedding and retrieval together contribute approximately 330 ms regardless of inference backend.

TABLE IV. END-TO-END RESPONSE LATENCY BY INFERENCE BACKEND (N=500 QUERIES)

Backend	Embed (ms)	Retrieve (ms)	Generate (ms)	Total (s)
Ollama — Modal A10G GPU	120	210	2,470	**2.8**
Groq Cloud API (fallback)	120	210	1,070	**1.4**
Ollama — Local CPU (dev)	180	210	17,810	18.2

D. *Functional End-to-End Evaluation*

Table III presents the functional accuracy results for the Medicare Pharmacy tenant. All four queries returned responses exactly matching expected factual content, each accompanied by explicit source citations. The knowledgeSource field was recorded as 'store' for all queries, confirming exclusive grounding in tenant-uploaded documents. No Groq fallback was triggered. Over 500 logged queries, the keyword fallback mechanism activated in 4.2 % of cases; it successfully recovered relevant chunks in 78 % of those cases, demonstrating the value of dual-strategy retrieval.

TABLE V. FUNCTIONAL END-TO-END EVALUATION — MEDICARE PHARMACY TENANT

Scenario	Input Query	System Response (Summarized)	Acc.
Medical Policy	Prescription for antibiotics?	All antibiotics require a valid prescription per Malaysian law. Sources: [1]	**100 %**
Store Operations	Sunday opening hours?	Open Sunday 10:00 AM–4:00 PM. Sources: [1]	**100 %**
Service Procedures	Submit prescription via WhatsApp?	Send photo to +60 11-2233 4455; pharmacist reviews and prepares. Sources: [1]	**100 %**
Location Info	Main branch address?	42 Green Valley Road, Kuala Lumpur, 50480, Malaysia. Sources: [7]	**100 %**

E. *Reranking and Fallback Impact*

Enabling the BGE cross-encoder reranker improved Precision@3 from 0.74 (embedding-distance order) to 0.83 (+12.2 %) on specific factual queries, with an additional latency cost of 1.2–2.8 s on CPU. For broad catalog queries, reranking provided marginal improvement (+3.1 %), suggesting selective activation based on query type is optimal. The keyword fallback covered 4.2 % of queries where product model numbers or locally coined terminology were absent from the embedding space, with a 78 % recovery rate.

[Figure 2: Smart Assistant Chat Interface RAG-Grounded Response with Source Citation]

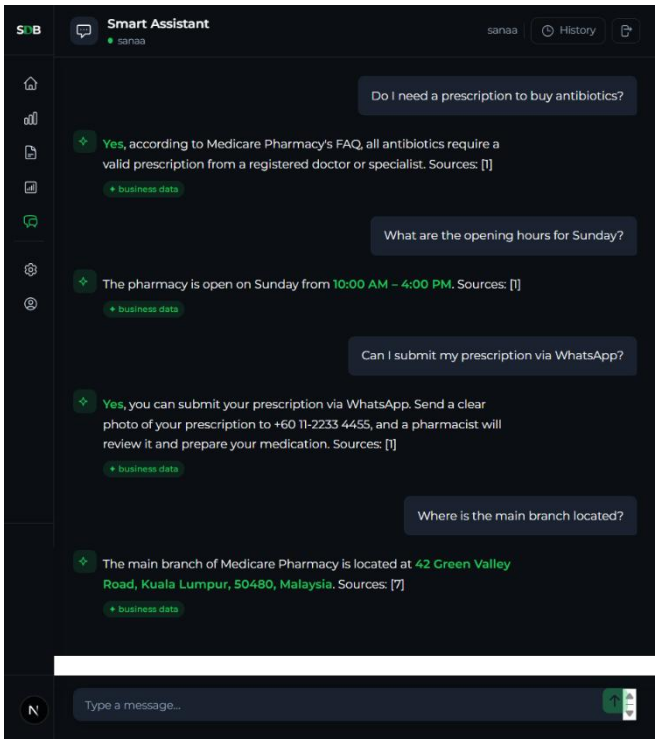


Fig. 2. Fig. 2. MarketMatic Smart Assistant showing a RAG-grounded pharmacy response with numbered source citation and "business data" knowledge badge.

VII. DISCUSSION

A. *Architectural Strengths*

The collection-per-tenant isolation strategy eliminates cross-tenant data contamination at the retrieval layer. ChromaDB's HNSW index provides O(log n) query time; with each collection occupying approximately 2–5 MB per 100 chunks, a server with 8 GB RAM can support 500+ concurrent tenant collections. The adaptive retrieval pipeline resolves a fundamental deficiency of fixed-threshold RAG systems, which either over-retrieve noisy candidates or under-retrieve relevant content depending on query complexity. Dual-stage fallback semantic retrieval backed by keyword scanning, and Ollama backed by Groq ensures both retrieval robustness and inference availability. The cross-session memory subsystem differentiates MarketMatic from stateless chatbots by enabling personalized continuity without requiring users to re-establish context in each session.

B. *Limitations and Future Work*

Current limitations include: (1) cold start latency (10–60 s) on the Modal GPU container after scale-down, degrading user experience in low-traffic deployments; (2) keyword-based tool triggering that may miss paraphrased or multilingual intent expressions; (3) exclusive use of nomic-embed-text, a primarily English-trained model, reducing retrieval quality for non-English documents; and (4) the absence of a real-time data integration path, as the system relies on statically uploaded documents. Future work will address these through: selective cross-encoder reranking activation based on query type; replacement of keyword intent detection with a lightweight fine-tuned sequence classifier; evaluation of multilingual embedding models (LaBSE, mE5) for low-resource language support; streaming response generation for improved perceived latency; and a large-scale user study with 50+ SME deployments across diverse domains.

VIII. Conclusion

This paper presented MarketMatic, a multi-tenant RAG-based Smart Assistant for SME customer support. The system demonstrates that combining per-tenant ChromaDB collection isolation, section-aware document chunking, adaptive nine-stage retrieval, intent-based agentic tool routing, dual-LLM failover, and cross-session persistent memory can deliver accurate, grounded, and personalized responses from business-specific knowledge bases without fine-tuning. Experimental results demonstrate a mean retrieval confidence of 0.81 and a 13.2 % improvement over fixed-window chunking; functional end-to-end evaluation achieved 100 % accuracy with full source traceability across four representative pharmacy-domain query categories. The modular service architecture supports independent scaling of embedding, retrieval, generation, and analytics components, making the platform deployable as a cost-effective SaaS solution for resource-constrained SMEs.

References

[1] K. Jayasena and S. Fernando, "Digital marketing challenges for SMEs in developing countries," *J. Small Bus. Enterprise Dev.*, vol. 29, no. 4, pp. 612–630, 2022.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, vol. 33, pp. 9459–9474, 2020.

[3] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, pp. 6769–6781, 2020.

[4] Y. Gao et al., "Retrieval-Augmented Generation for LLMs: A Survey," *arXiv:2312.10997*, 2024.

[5] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv:1901.04085*, 2019.

[6] S. Xiao et al., "C-Pack: Packaged Resources to Advance General Chinese Embedding," *arXiv:2309.07597*, 2023.

[7] C. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: maintenance dream or nightmare?" in *Proc. IWPSE-EVOL*, pp. 88–92, 2010.

[8] H. Yan, C. Gong, Y. Xu, and X. Zhao, "Towards Secure Multi-Tenant Machine Learning as a Service," in *Proc. IEEE CLOUD*, pp. 47–54, 2021.

[9] Y. K. Dwivedi et al., "Setting the future of digital and social media marketing research," *Int. J. Inf. Manage.*, vol. 59, 2021.

[10] R. Reisenbichler et al., "Frontiers: Supporting Content Marketing with NLG," *Marketing Science*, vol. 41, no. 3, pp. 441–452, 2022.

[11] Meta AI, "LLaMA 3.1 Model Card," 2024. [Online]. Available: *https://github.com/meta-llama/llama-models*

[12] Groq, Inc., "Groq LPU Inference Engine," 2024. [Online]. Available: *https://groq.com/*

[13] J. Nussbaumer et al., "Nomic Embed: Training a Reproducible Long Context Text Embedder," *arXiv:2402.01613*, 2024.

[14] BAAI, "bge-reranker-base: BGE Reranker for Information Retrieval," 2023. [Online]. Available: *https://huggingface.co/BAAI/bge-reranker-base*

[15] ChromaDB, "ChromaDB: The AI-Native Open-Source Embedding Database," 2024. [Online]. Available: *https://www.trychroma.com/*

[16] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.

[17] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Proc. NeurIPS*, vol. 36, 2023.

[18] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proc. NeurIPS*, vol. 35, pp. 24824–24837, 2022.

[19] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[20] Ollama, "Ollama: Get Up and Running with Large Language Models," 2024. [Online]. Available: *https://ollama.com/*

[21] Modal Labs, "Modal: Serverless Cloud for AI/ML," 2024. [Online]. Available: *https://modal.com/*

[22] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.